

A. Elements of Style

While the ultimate test of a program is how well it performs for the user, the value of the program is greatly limited if it is difficult to understand or update. For this reason, it is very important for programmers to write their code in the most clear and understandable way possible. We call this “programming style.”

Elements of Style

Perhaps the easiest way to explain coding style is this: give the bugs no place to hide. When our variable names are clearly and precisely named, we are leaving little room for confusion or misinterpretation. When things are always used the same way, then readers of the code have less difficulty understanding what they mean.

There are four components to our style guidelines: variable and function names, spacing, function and program headers, general comments, and other standards.

Variable and function names

The definitions of terms and acronyms of a software program typically consist of variable declarations. While variables are declared in more than one location, the format should be the same. Using descriptive identifiers reduces or eliminates the need for explanatory comments. For example, `sum` tells us we are adding something; `sumOfSquares` tells us specifically what we are adding. Use of descriptive identifiers also reduces the need for comments in the body of the source code. For example, `sum += x * x;` requires explanation. On the other hand, `sumOfSquares += userInput * userInput;` not only tells us where the item we are squaring came from, but also that we are creating a sum of the squares of those items. If identifiers are chosen carefully, it is possible to write understandable code with very few, if any, comments. The following are the University conventions for variable and function names:

Identifier	Example	Explanation
Variable	<code>sumOfSquares</code>	Variables are nouns so it follows that variable names should be nouns also. All the words are TitleCased except the first word. We call this style camelCase.
Function	<code>displayDate()</code>	Functions are verbs so it follows that function names should also be verbs. Like variables, we camelCase functions.
Constant	<code>PI</code>	Constants, include <code>#defines</code> , are ALL_CAPS with an underscore between the words.
Data Types	<code>Date</code>	Classes, enumeration types, type-defs, and structures are TitleCased with the first letter of each word capitalized. These are CS 165 constructs.

Function and program headers

It takes quite a bit of work to figure out what a program or function is trying to do when all the reader has is the source code. We can simplify this process immensely by providing brief summaries. The two most common places to do this are in function and program headers.

A function header appears immediately before every function. The purpose is to describe what the program does, any assumptions made about the input parameters, and describe the output. Ideally, a programmer should need no more information than is provided in the header before using a function. An example of a function header is the following:

```
/******  
 * GET YEAR  
 * Prompt the user for the current year. Error checking  
 * will be performed to ensure the year is valid  
 * INPUT: None (provided by the user)  
 * OUTPUT: year  
******/
```

A program header appears at the beginning of every file. This identifies what the program does, who wrote it, and a brief description of what it was written for. Our submission program reads this program header to determine how it is to be turned in. For this reason, it is important to start every program with the template provided at `/home/cs124/template.cpp`. The header for Assignment 1.0 is:

```
/******  
 * Program:  
 *   Assignment 10, Hello World  
 *   Brother Helfrich, CS124  
 * Author:  
 *   Sam Student  
 * Summary:  
 *   Display the text "Hello world" on the screen.  
 *   Estimated:  0.7 hrs  
 *   Actual:    0.5 hrs  
 *   I had a hard time using emacs.  
******/
```

General Comments

We put comments in our code for several reasons:

- To describe what the next few lines of code do
- To explain to the reader of the code why code was written a certain way
- To write a note to ourselves reminding us of things that still need to be done
- To make the code easier to read and understand

Since a comment can be easily read by a programmer and source code, in many cases, must be decoded, one purpose of comments is to clarify complicated code. Comments can be used to convey information to those who will maintain the code. For example, a comment might provide warning that a certain value cannot be changed without impacting other portions of the program. Comments can provide documentation of the logic used in a program. Above all else, comments should add *value* to the code and should not simply restate what is obvious from the source code.

The following are meaningless comments and add no value to the source code:

```
int i; // declare i to be an integer
i = 2; // set i to 2
```

On the other hand, the following comments add value:

```
int i; // indexing variable for loops
i = 2; // skip cases 0 and 1 in the loop since they were processed earlier
```

With few exceptions, we use line comments (//) rather than block comments (/* ... */) inside functions. Please add just enough comments to make your code more readable, but not so many that it is overly verbose. There is no hard-and-fast rule here.

“Commenting out” portions of the source code can be an effective debugging technique. However, these sections can be confusing to those who read the source code. The final version of the program should not contain segments of code that have been commented out.

Spacing

During the lexing process, the compiler removes all the spaces between keywords (such as int, for, or if) and operators (such as + or >=). To make the code human-readable, it is necessary to introduce spaces in a consistent way. The following are the University conventions for spaces:

Rule	Example	Explanation
Operators	tempC = 5.0 / 9.5 (tempF - 32.0)	There needs to be one space between all operators, including arithmetic (+ and %), assignment (= and +=) and comparison (>= and !=)
Indentation	{ int answer = 42; if (answer > 100) cout << "Wrong answer!"; }	With every level of indentation, add three white spaces. Do not use the tab character to indent.
Functions		Put one blank line between functions. More than one results in unnecessary scrolling, less feels cramped
Related code	// get the data float income; cout << "Enter income: "; cin >> income;	Much like an essay is sub-divided into paragraphs, a function can be sub-divided into related statements. Each statement should have a blank line separating them.

Other Standards

Because of the way printers and video displays handle text, readability is improved by keeping each line of code less than 80 characters long.

Subroutines and classes should be ordered in a program such that they are easy to locate by the reader of the source code. This usually means grouping functions that perform similar operations. For example, all input functions should be next to each other in a file, as should output functions.

Each curly brace should be on its own line; this makes them easier to match up.

Please make sure there are no spelling or grammatical errors in your source code.

Style Checklist

Comments

- program introductory comment block
- identify program
- identify instructor and class
- identify author
- brief explanation of the program
- brief explanation of each class
- brief explanation of each subroutine

Variable declarations

- declared on separate lines
- comments (if necessary)

Identifiers

- descriptive
- correct use of case
- correct use of underscores

White space

- white space around operators
- white space between subroutines
- white space after key words
- each curly brace on its own line

Indentation

- statements consistently indented
- block of code within another block of code further indented

General

- code appropriately commented
- each line less than 80 characters long
- correct spelling
- no unused (e.g. commented out) code

Examples

The following are two examples of programs with excellent programming style.

```
/* ****  
 * Program:  
 *   Homework 00, Add Integers  
 *   Brother Twitchell, CS 124  
 * Author:  
 *   Brother Twitchell  
 * Summary:  
 *   Demonstrates the amazing ability to add a positive integer and a  
 *   negative integer and to display the resulting sum.  
 **** */  
  
#include <iostream>  
using namespace std;  
  
/* ****  
 * Prompts the user for a positive and a negative integer.  
 * If required input is supplied, the two integers are added and the  
 * sum is displayed.  
 **** */  
int main()  
{  
    int positiveIntegerFromUser;  
    int negativeIntegerFromUser;  
    int sumOfIntegersFromUser;  
  
    // Prompt the user for a number  
    cout << "Enter a positive integer" << endl;  
    cin  >> positiveIntegerFromUser;
```

```

if (positiveIntegerFromUser > 0)
{
    cout << "Enter a negative integer" << endl;
    cin >> negativeIntegerFromUser;

    if (negativeIntegerFromUser < 0)
    {
        // amazing! we have both a positive and a negative integer
        // add them and output the results
        sumOfIntegersFromUser = positiveIntegerFromUser +
                                negativeIntegerFromUser;
        cout << "The sum of " << positiveIntegerFromUser;
        cout << " and " << negativeIntegerFromUser;
        cout << " is " << sumOfIntegersFromUser << endl;
    }
    else
    {
        // while the user has demonstrated his/her ability to enter a
        // positive integer, he/she failed to supply a negative
        // integer; give up!
        cout << negativeIntegerFromUser << " is not negative" << endl;
        cout << "Next time please enter a number less than zero (0)." << endl;
        cout << "Program terminating." << endl;
    }
}
else
{
    // the user has not supplied a positive integer; give up!
    cout << positiveIntegerFromUser << " is not positive" << endl;
    cout << "Next time please enter a number greater than zero (0)." << endl;
    cout << "Program terminating." << endl;
}
return 0;
}

```

Another example:

```

/*****
* Program:
*   Homework 00, Cube a Number
*   Brother Twitchell, CS 124
* Author:
*   Brother Twitchell
* Summary:
*   This program reads a number from a text file, cubes the number,
*   and outputs the result.
*****/

#include <iostream>
#include <fstream>
using namespace std;

/*****
* Returns the cube of the supplied integer value.
* Receives a pointer to the value to be cubed.
*****/
int cubedInteger(int number)
{
    // return the cube the supplied value
    return (number * number * number);
}

/*****
* Opens the input file and reads the number to be cubed. Outputs the
* original and cubed values. Closes the input file.
*****/
int main()
{
    int numberFromFile = 0;

    // open the input file, read a single integer from it, and close it
    ifstream inputFile("number.txt" /*filename containing the number*/);

    // yes, yes, I know we are not testing to see if we succeeded!
    //   This is only a short demonstration program.
    inputFile >> numberFromFile;
    inputFile.close();

    // output the original value and its cube
    cout << "Impress your date!\n";
    cout << "The cube of "
         << numberFromFile
         << " is "
         << cubedInteger(numberFromFile)
         << "."
         << endl;
    return 0;
}

```